HITAN UWB · DW1000 SS-TWR FIRMWARE

Consolidated Firmware Code Changes

110 kbps long-range migration, errata workarounds, the Rev-D receive-path fix, and multi-station UWB backoff — line-exact, grounded in the Decawave reference driver, DW1000 User Manual, and application notes.

Scope: Tag + Anchor `dw1000.cpp` / `.h` ; §8 also `main.cpp` , `uart_communication.c`

Reference tree: `documentation/fw_actual_ref/` (§8: 2026-09-23 anchor backup, `sC3.0.67`)

Scheme: SS-TWR · Ch 2 · PRF 64M · PLEN 2048 · PAC64 · code 9 **Date:** 2026-06-09 · **§8 added:** 2026-10-13

> **Read first — the Rev-D long-range problem is a RECEIVE problem, not a logic problem.** Bench A/B test: **Rev-D TX → Skylab RX works at 2 km+**, but **Skylab TX → Rev-D RX fails at ~400 m**. A directional fault rules out the TWR math, the protocol handshake, and timing (those would be symmetric). The fault is in Rev-D's receive chain. The leading firmware cause is **§3 (GPIO clocks never enabled → external RX LNA never switched in)** — verify it on the bench *before* assuming a hardware redesign.

Severity: BLOCKER ranging wrong/broken · IMPORTANT quality / range margin · OPTIONAL choice / verify. Each change cites the exact current line; "→" shows the replacement.

§1 110 kbps data-rate migration BLOCKER
§2 TX-1 errata — delayed-TX clock IMPORTANT
§3 Rev-D RX deafness — GPIO/LNA BLOCKER
§4 Long-range link-budget hardening IMPORTANT
§5 Robustness / verify OPTIONAL
§7 LDE algorithm tuning (weak-signal) EXPERIMENT
§8 Multi-station UWB backoff (two systems, one tag) BLOCKER
§6 Bench checklist + at-a-glance

§1 110 kbps data-rate migration

Switches the radio from 850 kbps to 110 kbps for greater RX sensitivity / range. Apply **identically on Tag and Anchor** — UM Table 21: the receiver must use the same data-rate/SFD config as the transmitter, or ranging stops entirely.

> **Preamble length: go to 2048, not 1024.** UM **Table 57 (Recommended preamble lengths)**: 110 kbps → **2048 or 4096** (850 kbps → 256/512/1024; 1024 is the *top of the 850k range*, below the 110k floor). For a max-range 110k system use **PLEN_2048**, which per UM **Table 6** requires **PAC64**. The firmware author already anticipated this — the commented block at `anc/inc/dw1000.h:133–144` is exactly DWT_PLEN_2048 / DWT_PAC64 / DWT_BR_110K. A longer preamble adds airtime (drives the timeout values up, below) and slightly lengthens the round-trip — still far under the ~67 ms 32-bit wraparound limit (1B), so it is safe; just keep the reply delays tight.

1A BLOCKER Carrier-integrator multiplier — `anc/src/dw1000.cpp:296–298` — the root-cause fix for negative clockOffsetRatio

```
/* current: 1024-sample constant used unconditionally */
clockOffsetRatio = dwt_readcarrierintegrator()
        * (FREQ_OFFSET_MULTIPLIER * HERTZ_TO_PPM_MULTIPLIER_CHAN_2 / 1.0e6);

/* → data-rate-aware: 110k accumulates over 8192 samples, not 1024 (8x larger) */
double freqOffsetMultiplier =
        (config.dataRate == DWT_BR_110K) ? FREQ_OFFSET_MULTIPLIER_110KB
                                         : FREQ_OFFSET_MULTIPLIER;
clockOffsetRatio = dwt_readcarrierintegrator()
        * (freqOffsetMultiplier * HERTZ_TO_PPM_MULTIPLIER_CHAN_2 / 1.0e6);
```

Basis: the two header macros differ only by divisor (1024 vs 8192); UM §7.2.40.11 states $N_{samples}$ = 8192 for 110 kb/s, 1024 otherwise . `config` is the file-static `dwt_config_t` from the header (in scope). 850k/6.8M correctly fall through to the 1024 macro — which is why 850k works today.

1B BLOCKER Reject non-physical ToF — `anc/src/dw1000.cpp:309–312 (you already removed abs())`

```
/* current (after your edit): negative tof now propagates as negative distance */
dwmInfo.tof = ((rtd_init - rtd_resp) / 2.0) * DWT_TIME_UNITS;
dwmInfo.distance = dwmInfo.tof * SPEED_OF_LIGHT * 100;
dwmInfo.dw_ready = VALID_DATA;

/* → guard: a negative result means a clock/wraparound glitch — drop it, don't report it */
double tof = ((rtd_init - rtd_resp) / 2.0) * DWT_TIME_UNITS;
if (tof < 0) {
    dwmInfo.dw_ready = DW_NO_VALID_DATA;        // non-physical: discard
} else {
    dwmInfo.tof = tof;
    dwmInfo.distance = dwmInfo.tof * SPEED_OF_LIGHT * 100;
```

```
    dwmInfo.dw_ready = VALID_DATA;
    dwt_readdiagnostics(&dwmInfo.diagnostics);  dwmInfo.wrong_ID = false;
}
```

At 110k the round-trip sits closer to the 32-bit timestamp wraparound ceiling (~67 ms). Once 1A is correct, negatives are rare glitches — reject them rather than fold to a fake short range (or, minimal: just restore `abs()`).

1C–1H BLOCKER Config + timeout constants — `both files, apply symmetrically`

#	LOCATION	CURRENT	CHANGE TO
1C	anc/inc/dw1000.h:126 tag/src/dw1000.cpp:47	`DWT_BR_850K,`	`DWT_BR_110K,`
1D	anc/inc/dw1000.h:121 tag/src/dw1000.cpp:42	`DWT_PLEN_1024,`	`DWT_PLEN_2048,` — UM Table 57 recommends 2048/4096 for 110k.
1E	anc/inc/dw1000.h:122 tag/src/dw1000.cpp:43	`DWT_PAC32,`	`DWT_PAC64,` — UM Table 6: preamble >1024 → PAC64.
1F	anc/inc/dw1000.h:128 tag/src/dw1000.cpp:49	`1100` (sfdTO)	`2049` — `(2048+1+64−64)`, 110k 64-symbol SFD at PLEN_2048. Any value ≥2049.
1G	anc/inc/dw1000.h:231	`RESP_RX_TIMEOUT_UUS 20000`	`25000` — its comment already says "preamble 2048"; bump for multipath margin.
1H	tag/Inc/dw1000.h:84	`TIMEOUT_PACS 1024/32*2`	`(2048/64*4)` — recomputed for PLEN_2048/PAC64; ×4 widens the preamble-detect hunt for weak long-range signal.

Auto-handled by `dwt_configure()` — no code: `SYS_CFG_RXM110K`, `LDE_REPC÷8`, `DRX_TUNE0b`, `DRX_TUNE1b_110K`, and `DRX_TUNE2` (indexed by PRF×PAC — picks up PAC64 automatically). **Optional:** for the more-robust Decawave 64-symbol SFD set `nsSFD` from `0` → `1` at anc/inc/dw1000.h:125 and tag/src/dw1000.cpp:46 (the author's commented 2048 block uses nsSFD=1) — must match on both ends (UM Table 21).

§2 TX-1 errata — force TX clock before delayed transmit

2A IMPORTANT Enable TX clock before each delayed `dwt_starttx` — `tag/src/dw1000.cpp:241 & anc/src/dw1000.cpp:411`

```
/* current: delayed TX issued without the errata workaround */
ret = dwt_starttx(DWT_START_TX_DELAYED);

/* → force TX clock on first (PMSC_CTRL0 bits 5,4 = 1,0) */
dwt_write8bitoffsetreg(PMSC_ID, PMSC_CTRL0_OFFSET,
    dwt_read8bitoffsetreg(PMSC_ID, PMSC_CTRL0_OFFSET) | 0x20);  // TXCLKS = 125MHz
ret = dwt_starttx(DWT_START_TX_DELAYED);
```

Errata §3.1.3 (verbatim): *"If the TX clock is enabled (PMSC_CTRL0 bits 5,4 set to 1,0 - force TX clock on) before the delayed TX command is issued then the problem does not happen."* The stock `dwt_starttx()` does not do this — it is the wrapper's job and is currently absent. A silently-failed delayed TX means the responder never replies → lost exchange; costs frames at the range edge where you can least afford them. Affects both SS-TWR response transmissions.

§3 Rev-D receive deafness — enable GPIO clocks for the external LNA

> **Leading suspected root cause of the Rev-D long-range failure.** `dwt_setlnapamode(...|DWT_LNA_ENABLE)` configures **GPIO6 = EXTRXE** to switch in the external RX LNA — but the firmware **never calls `dwt_enablegpioclocks()`**, and `dwt_setleds()` enables only the de-bounce clock (GPDCE), not the GPIO clock (GPCE/GPRN). UM §7.2.39: *"the GPIO clocks need to be turned on, before enabling or disabling the GPIO mode or value … by setting GPCE and GPRN in PMSC_CTRL0."* If the GPIO write doesn't take effect, EXTRXE never asserts, the Rev-D LNA is bypassed, and the receiver is deaf — exactly matching **RX-only, board-specific** (Skylab's integrated front-end doesn't depend on EXTRXE).

3A BLOCKER Add `dwt_enablegpioclocks()` after `dwt_setlnapamode()` — `tag/src/dw1000.cpp:113 & anc/src/dw1000.cpp:93`

```
dwt_setlnapamode(DWT_PA_ENABLE | DWT_LNA_ENABLE);
dwt_enablegpioclocks();  // REQUIRED: sets PMSC_CTRL0 GPCE|GPRN so EXTRXE (GPIO6) actually drives the LNA
```

One line, both files. APS004 §2: GPIO6/EXTRXE *"goes high when the DW1000 is in receive mode"* and (reference schematic) drives the RX-path RF switch into the LNA. This fix is curative **only if Rev-D's LNA is gated by EXTRXE** — confirm with §6 before concluding it is firmware-fixable vs a passive RX-front-end (matching/NF) hardware deficit.

Honest caveat: that an un-clocked `GPIO_MODE` write *silently* fails is documented in the UM but not independently re-confirmed by us; treat §3 as a high-probability hypothesis to verify on the bench (§6), not a settled fact. TX works at 2 km because the TX external-PA control (EXTPA/EXTTXE) is driven by the digital TX sequencer path, which is why the failure is RX-only.

§4 Long-range link-budget hardening

4A IMPORTANT **TX power temperature compensation** — `dwConfig, anc/src/dw1000.cpp:80–106 & tag/src/dw1000.cpp:102–136`

TX power is set once at boot via `dwt_configuretxrf()`; `dwt_readtempvbat()` / `dwt_calcpowertempadj()` are never called. APS023 Pt 1 §8.2: channel-2 TX power falls **~0.035 dB/°C** → a −10 °C→+30 °C outdoor swing ≈ **1.4 dB** of lost EIRP, straight off the long-range budget. **Fix:** periodically (post-wake / on temperature delta) read chip temperature and re-apply temperature-compensated power before TX.

4B IMPORTANT **Per-device antenna-delay calibration** — `anc/inc/dw1000.h:149–150 ; tag/Inc/dw1000.h:92–93`

Antenna delays are hardcoded (anc 16550/16500; tag 16500/16500). APS014: uncalibrated antenna delay ≈ **30 cm 3σ** error, drifting **2.15 mm/°C**. This is a *constant* offset (doesn't grow with range, won't stop you reaching distance) but it is your absolute-accuracy floor — calibrate per unit if sub-30 cm accuracy matters.

§5 Robustness / verify (lower priority)

#	LOCATION	CHANGE	SEVERITY
5A	`anc/src/dw1000.cpp:320–330` `tag/src/dw1000.cpp ~255`	Error path clears only `ALL_RX_TO|ALL_RX_ERR` before `dwt_rxreset()`, leaving "good-RX" latched bits set. Clear all relevant status bits. Runs constantly at long range (frequent RX errors).	MEDIUM
5B	`anc/src/dw1000.cpp:274 vs :160`	`dwRange` uses `frame_len == sizeof()` while `dwScanMode` uses `<=`. Make consistent (prefer `<=`). Robustness only — **not** the "drops all frames" critical bug an earlier pass mis-rated (ranging works at 3 km today, so `==` is functioning).	LOW
5C	`anc/src/dw1000.cpp:190`	Backoff uses `random()`, likely unseeded → every anchor shares one sequence → synchronized collisions in multi-anchor setups. Seed per-device (e.g. from device ID). **Promoted to 8E** now that two stations share a tag.	→ 8E

5E	`anc/src/usart.c:379 (RxEventCallback; no ErrorCallback exists)`
	`tag likewise`
		UART receive dies for good on any line error. Reception runs as `HAL_UARTEx_ReceiveToIdle_DMA` and is re-armed only from `UART_GetMessage` after a good idle event. In DMA mode the HAL treats *every* line error (framing, noise, overrun, parity) as blocking: it aborts the receive and calls `HAL_UART_ErrorCallback` — which this firmware does not define, so nothing re-arms. One glitch from a flaky USB adapter, a hot-plug, or a host opening the port at the wrong baud, and the anchor answers nothing until power-cycled (the main loop and watchdog keep running; only the UART is deaf). Bench 2026-10-13: right after one 3-byte Status frame was sent at the wrong baud (115200 instead of 921600), all three Mob anchors stopped answering at the correct baud, with no USB re-enumeration in the kernel log, and a power-cycle restored them. The field symptom is an anchor that "does not answer on USB" at BIT with its adapter enumerated fine.
		`void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart) { if (huart->Instance == USART6 || huart->Instance == USART3) { __HAL_UART_CLEAR_FLAG(huart, UART_CLEAR_OREF | UART_CLEAR_NEF | UART_CLEAR_FEF | UART_CLEAR_PEF); uartIdle = false; UART_RX_len = 0; HAL_UARTEx_ReceiveToIdle_DMA(huart, UART_Buff, sizeof(UART_Buff)); // re-arm: a line error ends one frame, not reception } }`
		The partial frame is dropped (the host times out and resends). Same for the tag's UART.
			BLOCKER

5D	`anc/src/dw1000.cpp:110–115`	LDE noise-threshold / peak-multiplier tuning for weak-signal first-path recovery — see **§7** (the `NEW_NTM` block is the hook, but its current value is a no-op; §7 gives the real values).	→ §7

§7 LDE algorithm tuning — weak-signal first-path detection

> **Do §3 (RX path) first.** LDE operates on signal that already reached the receiver. If Rev-D's LNA isn't switched in, no LDE value recovers a signal that is below the noise floor. Tune LDE *after* the receiver is actually hearing — to squeeze the last few dB of weak-signal margin, not as the cure for the 400 m failure. Hence EXPERIMENT, not blocker.

What the firmware runs today (verified): `dwt_configure()` calls the stock `_dwt_configlde()`, which writes `LDE_CFG1 = LDE_PARAM1 = PEAK_MULTPLIER(0x60) | N_STD_FACTOR(13) = 0x6D` (PMULT=3, NTM=13) and `LDE_CFG2 = 0x0607` for 64 MHz PRF. The `#ifdef NEW_NTM` block at anc/src/dw1000.cpp:110–114 is compiled

out — **and even if enabled it re-writes the same `0x6D`**, so it is a no-op-in-waiting, *not* the NLOS tuning its comment implies.

7A EXPERIMENT Lower the LDE first-path threshold (NTM + PMULT) — `write after dwt_configure(), e.g. anc/src/dw1000.cpp:110 / tag/src/dw1000.cpp ~118`

`LDE_CFG1` (reg 0x2E:0806) is one byte = **PMULT** (bits 7:5) + **NTM** (bits 4:0). Your current `0x6D` is tuned for *close-up line-of-sight accuracy* — the opposite of what a weak long-range first path needs.

LDE_CFG1	PMULT	NTM	MEANING (UM §7.2.47.2, VERBATIM GUIDANCE)
`0x6D` **(now)**	3	13	*"used … to give more accuracy in close-up LOS conditions."* Highest threshold — most likely to miss a weak/attenuated first path.
`0x6C`	3	12	Decawave default — *"reasonable compromise … between falsely triggering on noise peaks and missing real attenuated first paths."*
`0x07` **(NLOS)**	0	7	NTM: *"Where NLOS performance is more important … the default NTM value of 0x07 … would be a better choice."* PMULT: *"For NLOS operations these bits should be set to 0."*

```
/* after dwt_configure() — it calls _dwt_configlde() and would overwrite an earlier write */
/* sweep this value on the bench; lower = detects weaker first paths, but risks noise lock */
dwt_write8bitoffsetreg(LDE_IF_ID, LDE_CFG1_OFFSET, 0x07);  // PMULT=0, NTM=7 (NLOS / long-range)
```

Sweep, don't slam: `0x6D` (now) → `0x6C` (default) → `0x07` (NLOS). Measure first-path detection rate and range scatter at 400 m / long range at each. Too low a threshold makes LDE **lock onto noise** → false detects and garbage ranges → *more* effective loss. APS006 Part 2/3 is the tuning reference.

Placement: must be written *after* `dwt_configure()` (which sets the LOS default 0x6D), on the anchor that is the existing `NEW_NTM` location. Apply identically on Tag and Anchor. **Live-tunable from rf_analyzer ✓** — `LDE_CFG1` is at reg 0x2E, sub-offset 0x0806. The firmware WRITE opcode (14) carries a **16-bit** reg_offset (same struct as the READ opcode, whose ACC_MEM reads already use offsets >0xFF in production), so 0x0806 is reachable. `commands.UWB_write8bit_raw` now sends the offset as little-endian [lo,hi]; the analyzer Configuration tab has a "Live LDE tuning" control to sweep 0x6D/0x6C/0x07 in the field without re-flashing. (Earlier note said this needed a flash — corrected: only LDE_REPC math is driver-side; the byte itself is pokeable live, but resets to 0x6D on any dwConfig() re-run.)

Related, NOT a tuning knob: `LDE_REPC` (reg 0x2E:2804) must be divided by 8 at 110 kbps — `dwt_configure()` does this automatically (`reg16 >>= 3` for `DWT_BR_110K`). Leave it alone; do not hand-write it.

§8 Multi-station coexistence — UWB backoff (two anchor systems, one tag)

Line numbers in this section reference the anchor source in `firmware_code_for_reference/anchor-tag-bootload-backup-23-9-2026.zip` (`Anchor/Core/…`, firmware `sC3.0.67`), which is the build that carries opcode 19 (`BACKOFF_CTRL`). Added 2026-10-13.

Goal. Two stations with different `PC_ID` (e.g. Mob + Fly) range the *same* tag. The tag answers whichever anchor polls it (`tag/…/dw1000.cpp:253` checks only `des == SELF_ID`, no pc_id filter), so the only shared resource is the air. The anchor already implements a passive **listen-before-talk**: between UART commands it sits in `dwScanMode()` (main.cpp:408, gated by `BackOFF_CTRL`); any frame whose pc_id (`src>>2` for anchor frames, `des>>2` for tag frames) is foreign arms `backoff_timer = now + random()%200 + 10` (dw1000.cpp:201, i.e. 10–209 ms); a `SEARCH_TWR` that arrives while it is armed skips `dwRange()` and answers with `errors.bit4 = UWB_Backoff` (main.cpp:252, uart_communication.c:176). The PC side will treat that bit as "not polled" and yield. The mechanism is sound **but as written it starves the second station** — §8A.

Starvation (why 8A is a blocker). dw1000.cpp:201 *re-arms* the timer on **every** foreign frame. Station A ranging at ~30 Hz puts a foreign frame on the air every ~11 ms at B's anchors; each one pushes B's deadline ≥10 ms further out, so B's backoff *never expires* (P(random ≤ 11 ms) ≈ 1%). B reports `UWB_Backoff` forever, never transmits, so A's anchors never hear B and A never yields. Whoever is already running wins permanently — in both directions, after any handover. A passive backoff can only hand the channel over if the waiting side is *guaranteed* to transmit after a bounded wait (classic CSMA). Fix: arm once per busy period, expire, transmit.

Bench result, firmware sC3.0.69 (2026-10-13, Fly PC id 1 + Mob PC id 11, tag 56).

- **8B works:** opcode 19 = 0 releases an armed backoff at once and the anchor ranges on the next request; re-enabling does not re-arm by itself.
- **Arming works:** five rounds of the other station's traffic arm all three anchors.
- **Expiry is broken — blocker:** the Status backoff field (now ms left) counts down and **stops at 1**; the anchor answers every SEARCH_TWR with bit 4 for good (watched 3 s, and indefinitely in earlier runs). One burst of foreign traffic silences a station permanently. Likely an off-by-one in the countdown or in the expiry compare (`<` vs `<=`, or a decrement that stops at 1).
- **8C half done:** Status carries ms left; the TWR reply's distance field is still 0 on a backed-off frame, so the PC cannot size its yield.

PC-side workaround in place: an anchor that reports backoff for longer than 250 ms (+0–150 ms jitter) is released with opcode 19 off/on and polled again. Measured, both stations on one tag for 20 s: **42 + 30 valid ranges/s** (each alone: 81–89/s), p95 gap 83 / 229 ms; with backoff off: 13 + 10/s, p95 gap 431 / 368 ms, max 3.6 s. Field scenario (one station alone 10 s, the other joins for 20 s, the first stops), both orders: the newcomer's first range after 0.4–1.4 s; while both run 21–48 valid ranges/s each, worst gap 0.7–1.9 s; the station left alone is back to full rate (85–90/s) within the first second. Releasing a whole station at once instead of per anchor measured worse (worst gaps 5–6 s). With expiry fixed, the release becomes the rare §8A bounded wait instead of the normal path, and the fine window of 8D can interleave the stations per exchange, which is what will shorten those overlap gaps.

Bench result, firmware sC3.0.70 (2026-10-14, same rig). Expiry fixed: after a burst of the other station's traffic the Status field read 42–44 ms left, counted down to 0, and the anchor ranged again by +134 ms. **TWR reply:** carries the backoff bit only, by design (no time field; only Status counts down). That is all the PC needs: it polls every round anyway, so the next reply shows when the backoff has ended. Measured (both stations ranged lower alone than the day before, 60–68/s, so compare within the day): simultaneous start 30.6 + 31.5 valid ranges/s, p95 gap 111 / 90 ms (backoff off: 14.1 + 8.7/s); join/leave both orders — newcomer's first range 0.4–0.75 s, 27–45/s each while both run, worst gap 0.8–1.9 s, full rate again within 1 s. A 30 ms PC yield was tried and was worse (fewer whole rounds). The remaining overlap gaps are slot-like alternation; the fine window of 8D is the next firmware lever. (PC side since: taking turns, ON/OFF 250/250 — Mob's share 0.3–1.1 → 7.3–8.0 valid ranges/s with Mob ranging from its front anchor only.)

8A BLOCKER Arm the backoff once per busy period; own-pc_id frames must not clear it — `anc/src/dw1000.cpp:198–202 (dwScanMode)`

```
/* current: every foreign frame re-arms (never expires under continuous foreign traffic);
   every OWN frame clears it (A's own next poll erases the yield B just earned) */
if (dwmInfo.detec_pc_id == true){
    dwmInfo.backoff_timer = 0;
}else{
    dwmInfo.backoff_timer = _1ms_tick + random() % TIME_TOP_RANGE_MS + MIN_BACKOFF_MS;
}

/* → arm ONLY when not already running; a running timer is left alone by both own and foreign frames.
   "Running" = deadline still in the future; an expired deadline counts as idle so it can re-arm on the next busy period. */
bool running = (dwmInfo.backoff_timer != 0) && (dwmInfo.backoff_timer > _1ms_tick);
if (!dwmInfo.detec_pc_id && !running) {
    dwmInfo.backoff_timer = _1ms_tick + random() % TIME_TOP_RANGE_MS + MIN_BACKOFF_MS;
}
```

Bounded wait: at most `MIN_BACKOFF_MS + TIME_TOP_RANGE_MS − 1` after the first foreign frame, the next `SEARCH_TWR` goes out regardless (main.cpp:252 already clears the timer on expiry and ranges). That poll is what the *other* station's anchors hear, back off from, and report to their PC, which then yields. Without the own-pc_id change, an anchor that hears a sibling anchor's poll (same pc_id, every round) drops a foreign backoff in progress, so a running station never actually yields even once its PC wanted to. Collision risk on the barge-in poll is ~30% (air occupancy of one station ≈ 3×(3–4 ms) per 33 ms round); a collided poll is a normal miss and the next one goes out after the next backoff.

8B BLOCKER `BACKOFF_CTRL = 0` must also release an armed timer — `anc/src/main.cpp:281–283 (UART_OPCODE_BACKOFF_CTRL)`

```
/* current: disables the scan only — an already-armed deadline keeps blocking SEARCH_TWR (main.cpp:252) until it expires */
if(msg.op_backoff_ctrl_req.BackOFF_CTRL == 0){
    BackOFF_CTRL = false;
    dwmInfo.backoff_timer = 0;   // → release: "disable backoff" means "poll on the next SEARCH_TWR"
}
```

This is the PC's **barge-in** lever: after N consecutive `UWB_Backoff` replies the PC sends opcode 19 = 0 to one anchor, polls once (making the other station yield), and sends 19 = 1 again. With 8A in place barge-in is rarely needed; without 8A it is the *only* way for the second station to get in, and today it costs an extra ≤209 ms wait because the armed deadline survives the disable. One line.

8C below is dropped (2026-10-14): the PC uses only the backoff bit of the TWR reply and needs no time — it polls every round, so the next reply shows when the backoff ended. sC3.0.70 already clears the bit on expiry and counts down in Status. Kept for the record.

8C DROPPED Report backoff *time left* (ms), not a raw tick, and only while it is running — `anc/src/uart_communication.c:176 (TWR reply) · :193 (Status) · :108 / :128 (BIT, wake-up)`

```
/* current (TWR reply): bit = "timer non-zero" — also set by a STALE, already-expired deadline that no SEARCH_TWR has cleared yet;
   uwb_distance on a skipped range carries the PREVIOUS exchange's distance; Status.UWB_backoff is the absolute _1ms_tick deadline (meaningless to the PC) */
reply->op_twr_rply.errors.bits.UWB_Backoff = !!dwmInfo.backoff_timer;

/* → one helper, used by all four replies */
static uint32_t backoff_left_ms(void) {
    return (dwmInfo.backoff_timer > _1ms_tick) ? (dwmInfo.backoff_timer - _1ms_tick) : 0;
}
/* TWR reply (:176): */
uint32_t left = backoff_left_ms();
reply->op_twr_rply.errors.bits.UWB_Backoff = (left != 0);
if (left) reply->op_twr_rply.uwb_distance = left;   // skipped range: distance field carries ms-left instead (PC reads it only when bit4 is set)
/* Status reply (:193): */
reply->op_stt_rply.UWB_backoff = backoff_left_ms();
/* BIT (:108) and wake-up (:128) replies: */
reply->…errors.bits.UWB_Backoff = (backoff_left_ms() != 0);
```

Wire-compatible: no struct change, no length change; `uwb_distance` is `uint32_t` and otherwise 0/stale on a skipped range (`data_status` already says no DW data). The PC then waits exactly `left` ms (+ a few ms) instead of a blind 300 ms, and never mistakes a stale deadline for a live backoff. Keep the existing convention that the reply is still sent (the anchor must answer every `SEARCH_TWR`, backoff or not).

8D CHOOSE Backoff window: coarse (TDM-like, 300 ms slots) or fine (exchange-level interleave) — `anc/inc/dw1000.h:93–94`

MODE	CONSTANTS	BEHAVIOUR WITH TWO STATIONS	PC SIDE
Coarse (today's values)	`TIME_TOP_RANGE_MS 200` `MIN_BACKOFF_MS 10`	Stations alternate in bursts: one ranges at ~30 Hz until the other barges in (≤209 ms after it first hears it), then yields. Each station sees ~9-round holes of ~300 ms per handover.	On bit4: wait the PC's yield (Advanced, default 300 ms; 40 ms while taking turns) and poll again; a long backoff is released via 8B.
Fine (recommended)	`TIME_TOP_RANGE_MS 15` `MIN_BACKOFF_MS 1`	Backoff on the scale of one exchange (~3–4 ms air, ~11 ms with UART). Stations interleave per exchange; each gets ~12–15 Hz with jitter and no holes. No handover protocol, no starvation, no yield timer. **Requires 8A** (with re-arm, a short window makes starvation worse, not better).	On bit4: re-send `SEARCH_TWR` to the same anchor after ~5 ms, up to 3 tries (a rejected request costs ~1 ms of UART), then move on.

The PC's position/bearing filters propagate with the measured `dt` (CV predict, dt-scaled gates), so either mode is safe for the filtering; the fine mode is better for it (jittered 15 Hz beats 30 Hz with 300 ms holes — fewer lost rounds, less lag at every handover). Both values are compile-time; a runtime setter (opcode 19 payload byte 2 = top-ms) would let the PC pick per deployment.

8E IMPORTANT Seed `random()` per anchor — `anc/src/main.cpp init (before the main loop); promotes 5C`

```
srandom((unsigned)(PC_ID ^ (dwmInfo.anchor_id << 8) ^ HAL_GetTick() ^ HAL_GetUIDw0()));
```

With two stations this is no longer cosmetic: all six anchors run the same unseeded sequence, so two anchors that hear the same foreign frame pick the same backoff and their barge-in polls collide deterministically, round after round. Any per-device seed breaks the lock-step.

8F IMPORTANT Imposter mode (BIT) must match the id a poll can actually carry — bit 15 masked — `anc/src/dw1000.cpp:402 (dwTagMode) · :40 (dwConfig, tx_resp_msg.src) · :232 (dwRange, rx_resp_msg.src) · main.cpp:248`

> **Latent: no station is known to hit this today.** It was suspected for Mob's failing BIT on 2026-10-13, but Mob's PC id read back as 11 (self ids 0x2C–0x2E), well clear of it. The station self-test switches each anchor to imposter mode and has the others range it under its Status `self_id = PC_ID<<2 | anchor_id`. `SEARCH_TWR` strips bit 15 of the requested id as the LoRa-only flag (main.cpp:248, `& 0x7FFF`), so the poll goes out with `des = self_id & 0x7FFF` — but the imposter accepts a poll only when `des == self_id` *unmasked*. Any station whose **PC id is ≥ 8192** (bit 15 of `self_id` set) would have anchors that can never be ranged as imposters. Keep every station's PC id below 8192.

```
/* current (dwTagMode): the imposter answers only its full 16-bit self id */
if (rx_buffer.des == tx_resp_msg.src) {

/* → the id a poll can carry: bit 15 is the LoRa-only flag on the UART side, never part of an over-the-air id */
if (rx_buffer.des == (tx_resp_msg.src & 0x7FFF)) {

/* and in dwRange, the expected responder id is already the masked tag_dest (rx_resp_msg.src = dwmInfo.tag_dest) —
   the imposter's reply must carry the same, so build its response src masked too: */
tx_resp_msg.src = ((dwmInfo.mine_pc_id << 2) | dwmInfo.anchor_id) & 0x7FFF;   // dwConfig, :40
```

Two masks, both sides of the same exchange; a real tag is unaffected (its id never has bit 15). **Until this lands**, the field workaround is a PC id below 8192 on the affected station (`SET_ID`, hitan-cli menu 2) — the PC-side BIT now names this condition from the Status `self_id` instead of reporting three unrangeable pairs. Longer term, bit 15 of the UART id should not double as a flag at all: 14 bits of PC id + 2 of anchor id already fill the 16, so the LoRa-only request belongs in its own byte.

8G IMPORTANT SET_ID must also refresh the UWB frame ids, or say "reboot required" — `anc/src/main.cpp (case UART_OPCODE_SET_ID) · dw1000.cpp:36–40 (dwConfig)`

The changelog (C1.6.03) says a PC-id change no longer needs a reset. That holds only for the Status `self_id`: the handler calls `SetAnchor_self_id()`, which rebuilds `lora_rxInfo.self_id`. The UWB side keeps the boot-time value — `dwmInfo.mine_pc_id` is set once by the static initializer, and `tx_poll_msg.src`, `rx_resp_msg.des` and `tx_resp_msg.src` are built from it once in `dwConfig()`. Until a power-cycle the anchor **reports the new id but ranges, answers BIT as imposter, and judges backoff under the old one**: a BIT run straight after SET_ID polls the imposter at an id it does not answer, and every pair fails.

```
/* SET_ID handler, after a successful Flash_Write: */
SetAnchor_self_id();
dwmInfo.mine_pc_id = PC_ID;
uint16_t id = (dwmInfo.mine_pc_id << 2) | dwmInfo.anchor_id;
tx_poll_msg.src = rx_resp_msg.des = tx_resp_msg.src = id;   // the three frame templates dwConfig() fills
```

Also on the PC side: the payload is the PC id itself (the anchor id comes from the cable pins). The hitan-cli SET_ID menu used the older `pc<<2 | anchor` packing, so entering "PC 2, anchor 3" stored PC id 11; fixed 2026-10-13, and it now prompts for a power-cycle.

PC-side contract (hitan_station), for reference. `errors` bit4 (0x10) on a TWR reply means *not polled*: not a missed range, not a dead anchor, not a silent tag (no LoRa wake-up, no spare engagement, no miss count). On bit4 the PC waits its yield (Advanced, default 300 ms; 40 ms while taking turns) and polls again — the next reply shows whether the backoff has ended; an anchor backed off past the release time is freed via opcode 19 off/on (8B). Stations must have distinct `PC_ID` (bits 15:2 of `self_id`, set with `SET_ID`) — verify on both stations before the first joint test.

§9 Attributable CIR, per-exchange timing, negotiated TX power (host request, 2026-10-21)

Line numbers in this section reference the current anchor/tag source (`Anchor/Core/Src/dw1000.cpp`, `Anchor/Core/Lora/Src/Lora.cpp`, firmware anc3.1.5). Host side: `hitan_rust` on the same date.

Why. The field failure that matters is close range with no line of sight — a MOB station near the ground, the tag behind bushes or rocks. Both radios misbehave for one reason: a weak direct path and a strong reflected one. **LoRa:** the receiver sits near compression (above ≈−30 dBm), the level differences between anchors that carry the direction are squashed, and two-ray fading off the ground swings each level ±8 dB within seconds. **UWB:** the first path is weak against the reflections, the accumulator saturates on the strong path, and the latched diagnostics the host gates on stop describing the leading edge; bearings scatter 12–14° per round at 1 m (bench replay 2026-10-21). What the host can do is bounded by what the anchor reports. Today only `dwt_readdiagnostics` is latched per valid exchange (`dw1000.cpp:355`, `dwRange`); `LDE_PPINDX` and `ACC_MEM` are read live over opcode 12 after RX has re-armed, so they belong to whatever frame came next — the host withholds prNlos for that reason. Measured on the truth-labelled recordings (`examples/nlos_features.rs`): the three latched first-path taps carry **no** NLOS information (AUC 0.46); RX−FP is the only discriminator and it is weak (AUC 0.62–0.76).

Rule zero for everything in §9: the link is never lost. Losing communication with the tag is the one failure that is not allowed; every adaptive change below is built so that the worst case of a bad decision, a missed frame or a crash mid-negotiation is a few hundred milliseconds on the robust profile — never a tag that cannot be reached.

- **Control plane stays robust, always.** The anchor→tag LoRa command path (wake-up, poll, profile and power requests) is fixed at maximum power and the robust spreading factor, and the tag's LoRa receiver stays on that configuration. Only the tag's *beacon* and the UWB exchange adapt. LoRa and UWB are separate radios on both sides, so the LoRa control channel is a permanent back door: any state can be undone from it.
- **Profile 0 is the ground state.** Any doubt on either side → profile 0, unilaterally and at once. Tag: no frame from any anchor for 1 s. Anchor: two consecutive polls unanswered. Power-up, reset and watchdog restart land on profile 0. Wake-ups always go at profile 0.
- **Make-before-break.** A move to a *less* robust profile or a lower power needs the tag's echo of the request *and* the anchor's confirmation of that echo (three frames) before either side switches, and two consecutive frames with the level inside the target band after the switch before the next step. A move to a *more* robust profile or higher power takes effect immediately and without confirmation.
- **Rate-limited and bounded.** At most one step every 2 s away from robustness; no limit toward it. Power never below the level that leaves 10 dB of margin over the receiver's sensitivity at the current range, measured at the anchor, not predicted.
- **Channel changes are opt-in and proven first.** Shipping builds keep UWB channel 4 disabled until the bench shows ≥ 80 % valid exchanges at 10 m with the installed antennas; even then the near profile falls back to channel 2 after one missed exchange.
- **Chaos test, before any of §9D/9H ships.** 100 trials each of: walk the tag away at a run during a step down; cover the tag's antennas by hand; power-cycle one anchor mid-negotiation; power-cycle the tag; two stations on one tag. Pass criterion: the tag is reachable again within 1 s in every trial, zero cases of permanent loss. One failure blocks the release.

Host side, no firmware needed. (1) **LDE NLOS preset:** `LDE_CFG1` is live-tunable via opcode 14 (`dw1000.cpp:22`, default 0x6D = LOS); APS006 Pt 2 recommends NTM 7 / PMULT 3 → `0x67` for NLOS — offered as a LOS/NLOS knob, not a default (more false first paths in clean air). (2) **Level-dependent range bias (APS011):** the received level is in the diagnostics, so the bias table is applied on the host.

9A Latch the peak-path index and the exchange tick with the diagnostics important

In `dwRange()`, right after `dwt_readdiagnostics(&dwmInfo.diagnostics)` on a valid range, read into the same latched block: `LDE_PPINDX` (reg 0x2E, sub 0x1000, 2 bytes), `RXPACC_NOSAT` (reg 0x27, sub 0x2C, 2 bytes) and `HAL_GetTick()`. Append to the **DIAGNOSTIC_STATUS (opcode 13)** reply after `txw` (bytes 42–43):

BYTES	FIELD	TYPE
44–45	`lde_ppindx`	u16 LE
46–47	`rxpacc_nosat`	u16 LE
48–51	`exchange_tick_ms`	u32 LE, tick at the end of the exchange

Older hosts ignore the extra bytes; the new host accepts the shorter reply from older firmware. **Host use:** prNlos per APS006 Pt 3 §5.2 from `|FP_INDEX/64 − PP_INDEX|`, attributable at last; the SFD correction on RXPACC; the anchor's tick replaces the host receipt time in the per-exchange alignment.

9B Latch a CIR window around the first path, new opcode 21 `READ_CIR_WINDOW` important

In the same latched block, 32 complex accumulator samples starting 8 before the detected first path, read while the accumulator still holds this reception (before RX re-arms):

```
start = (FP_INDEX >> 6) - 8;                /* FP_INDEX is Q10.6 in RX_TIME */
_dwt_enableclocks(READ_ACC_ON);
dwt_readaccdata(buf, 1 + 4*32, start * 4);   /* 1 dummy octet, then (i16 re, i16 im) x 32 */
_dwt_enableclocks(READ_ACC_OFF);
```

Reply to opcode 21 (no request payload): bytes 2–3 `fp_index` u16 (raw Q10.6), 4–5 `start` u16, 6 `n` u8 (32), 7… `n` × (i16 re, i16 im) LE — 135 bytes, ≈1.5 ms at 921600. A separate opcode so the per-round cost is opt-in: the host reads it only with the quality gate on, right after the diagnostics. **Host use:** the leading-edge features the UWB NLOS literature actually uses (rise time, energy before vs after the first path, peak-to-first-path delay, kurtosis) and later a small classifier on the truth-labelled recordings. This is the only change that makes a real CIR-based NLOS gate possible.

9C Per-exchange data in the SEARCH_TWR reply important

Append to the **SEARCH_TWR (opcode 5)** reply after byte 16 (`anchor_error_v`):

BYTES	FIELD
17–20	`exchange_tick_ms` u32 LE, `HAL_GetTick()` when the response frame was received
21	`tag_lora_tx_dbm` i8, from the tag's frame (9D); 0x7F = unknown
22	`tag_uwb_tx_level` u8, from the tag's frame (9D); 0xFF = unknown
23	`lora_signal_rssi` u8, SX126x `SignalRssiPkt` , encoded like `lora_rssi` (raw = dBm + 256)

`lora_rssi` (byte 11) is `RssiPkt` today. `SignalRssiPkt` is the level of the LoRa signal after despreading — it excludes in-band noise and interference, which is exactly what differs near the ground. Both come from the one `GetPacketStatus()` call in the LoRa receive path (`Lora.cpp`). The host's direction maths moves to the signal level.

9D Negotiated TX power, tag and anchor (AGC-style) important

Goal: keep every receiver in its linear range at any distance — LoRa level differences stay meaningful close in, the UWB accumulator does not saturate. **Start at today's maximum** and step down only as the tag comes close; back to maximum the moment the tag goes quiet.

- **Loop on the anchor** (not the host: it must react within a few packets, and must not depend on the UART). LoRa target: `SignalRssiPkt` at the anchor in −70 ± 6 dBm. UWB target: `rx_power_dbm` (from the diagnostics) in −80 ± 5 dBm — flat on the APS011 bias curve; above −65 dBm the DW1000 saturates.
- **Steps:** one per packet; 2 dB for LoRa (`SetTxParams` , `Lora.cpp:249` , today fixed at `TX_OUTPUT_POWER`); one level for UWB from a table of 8 `TX_POWER` register values 3 dB apart, from the current maximum (`dw1000.cpp:134`) downward; smart TX power stays disabled. Dead band: no step inside the band.
- **Verification:** the next frame from the tag carries the power it actually used (below); a mismatch for two frames resets the request to maximum.
- **Floor and fallback (rule zero):** never request below −9 dBm LoRa or the lowest UWB table entry, and never below 10 dB of measured margin; after 1 s with no frame from the tag, request maximum again, and the tag returns to maximum by itself after 1 s without hearing an anchor; wake-up packets and all anchor→tag LoRa commands always go at maximum. A lost tag is found at full power first.
- **Three anchors, one tag:** the tag obeys the most recent request; one station's anchors share one target so their requests agree within a step; the front anchor (the only UWB ranger on MOB) owns the UWB request.
- **Over the air:** the anchor's UWB poll and LoRa wake/poll frames carry `req_lora_tx_dbm` (i8) and `req_uwb_tx_level` (u8); the tag's response frames carry the two it used. Older tags ignore the request and report 0x7F/0xFF, forwarded unchanged (9C), so the host can tell.
- **Host:** switched and configured through opcode 22 `SET_LINK_FEATURES` (9I), applied by the station worker like the backoff toggle; the current request and the tag's echo in the **STATUS** reply. The host normalises the hot/cold bar and the LoRa calibration by the reported tag power (`level + (max − reported)`) so a quieter tag does not read as a farther one. Default off until the field test says otherwise.

9E LoRa frequency diversity phase 2

Near the ground the level at one frequency sits in a fade for seconds. The tag alternates two channels by packet-counter parity, the anchors listen in step, the host averages direction across the pair. A bigger change (channel plan, per-channel duty cycle, both firmwares); only if 9D leaves the arc unstable near the ground.

9F Repeat the exchange and report the minimum important

Anchor only, no negotiation. NLOS error is one-sided: a reflection only ever makes a range read *long*. On a SEARCH_TWR the anchor runs `K` exchanges back-to-back and reports the **shortest** valid range (and that exchange's diagnostics). `K` adapts to the anchor's own last diagnostics: 1 while RX−FP is under the soft threshold, 3 when it is above. Cost ≈11 ms per extra exchange; the station worker already tolerates longer rounds. Request byte: SEARCH_TWR payload gains an optional third byte `k_max` (0 = anchor decides, 1–5 = fixed); reply byte 24 `k_used` . **Host use:** none required — the range is just better; the host reads `k_used` for the round-time bookkeeping and the per-exchange tick (9C) of the reported exchange. The cheapest large NLOS win on this list.

9G LDE preset per state important

Anchor only. The anchor switches `LDE_CFG1` itself: NLOS multiplier (NTM 7 / PMULT 3, `0x67`) while its last RX−FP sits above the soft threshold, LOS (`0x6D`) when it clears, with a dead band of 2 dB and a hold of 10 exchanges so it does not chatter. Requires the RX reset the driver notes after an LDE

change (`dw1000.cpp:234`). The host keeps its opcode-14 override; a STATUS bit reports which preset is active. Pairs with 9F: when the anchor flips to the NLOS preset it also raises `K` .

9H Link profile negotiation (one byte, not six parameters) important

Everything the tag must change in step with the anchor — TX power (9D), frequency diversity (9E), and the radio settings below — is negotiated as **one profile index**, with the request / echo / confirm / fall-back state machine of rule zero. The anchor picks the profile from RX level and range; the tag echoes it in every frame; both drop to profile 0 after 1 s of silence; wake-ups and all anchor→tag LoRa commands always go at profile 0. A profile only ever changes the tag's beacon and the UWB exchange, never the LoRa control path.

PROFILE	UWB	LORA	POLLS (9F)	WHEN
0 far (today)	ch 2, 110 kbps, 2048 preamble, PRF 64, max power	SF high, slow beacon, max power	1	default; any silence
1 mid	ch 2, 850 kbps, 1024 preamble, power stepped (9D)	SF mid, beacon 2×, power stepped	1–3	RX level in band, range < ~200 m
2 near	ch 4 if the antennas allow (900 MHz bandwidth: paths 1 ns apart resolve instead of smearing into one first path), else ch 2; 6.8 Mbps, 256 preamble; power low	SF low, fast beacon, power low; channel alternation (9E)	3	RX level hot, range < ~30 m

- **What each setting buys.** Bandwidth (ch 4/7) is the one real multipath lever on the DW1000 — it needs a bench check that the installed antennas work there at all. The faster data rate and shorter preamble do not fix NLOS by themselves; they buy the repeats of 9F and more rounds per second in the same time. LoRa SF and beacon rate buy more packets per second close in, which is what steadies the arc (direction is an average over packets), and battery far out. PRF stays at 64 MHz (already the NLOS-friendly choice); coding rate changes nothing.
- **Over the air:** one byte `profile_req` in the anchor's poll and wake frames, one byte `profile_used` in the tag's frames; forwarded to the host in the TWR reply (byte 25). Older tags report 0xFF and stay on profile 0.
- **Host:** opcode 22 (9I) carries `profile_max` and `profile_override` ; the Configuration tab shows the requested and echoed profile per anchor. The host's time alignment (9C) and quality gates are profile-agnostic; the per-anchor first-path SNR history (relative gate) must reset on a profile change, since the normal level moves with the preamble.
- **Host side, no firmware:** the SS-TWR clock-offset correction (range error ∝ reply delay × crystal offset; the offset is already in the diagnostics as DRX_CAR_INT) is applied on the host.

9J Change handshake: the tag is told, acknowledges, and nothing switches until both sides have proof required

Every change the tag must apply — power, profile, channel alternation — goes through one three-way handshake with a sequence number. No side ever switches on a request alone, and no side is ever on a configuration the other does not know about.

STEP	FRAME	CARRIES	WHO SWITCHES
1 request	anchor → tag, in the UWB poll *and* in the LoRa command frame (max power, robust SF)	`seq` , requested `profile` , `lora_dbm` , `uwb_level`	nobody
2 acknowledge	tag → anchor, in its UWB response *and* its next LoRa beacon	`ack_seq` = the request it will apply, plus its *current* settings	nobody — the tag still runs the old settings
3 confirm	anchor → tag, next poll and LoRa command	`confirm_seq` = the acknowledged request	tag switches on receipt; anchor switches its receiver on sending, and keeps the old receiver configuration in its listen rotation (below) until the first frame arrives on the new one
4 proof	tag → anchor on the new settings	`profile_used` , powers used	step complete; the next step may start after `step_interval`

- **Sequence numbers.** One byte, incremented per request; a request with a stale or repeated `seq` is ignored by the tag; an ack for a `seq` the anchor did not send is ignored by the anchor. Every frame carries the sender's current settings, so either side can see a mismatch at any time.
- **Retries.** The anchor repeats a request on every poll until acknowledged, up to 5 polls; then it cancels (the tag never switched, nothing to undo). The anchor repeats the confirm until the first frame arrives on the new settings, up to 5 polls; then both sides revert (next bullet).
- **Revert without a word.** If the tag applied a confirm and then hears nothing on the new settings for 500 ms, it reverts to the *previous* settings on its own, and to profile 0 after 1 s total. The anchor, on 2 missed polls after a confirm, goes back to the previous settings for 2 polls and then to profile 0. Both reverts are unilateral and need no frame, because the failure case is exactly that no frame gets through.
- **Listen rotation (one radio per side).** The DW1000 holds one configuration at a time, so during a transition the anchor alternates its receiver between the new and the old configuration poll by poll until proof arrives, and the tag, when idle, alternates between its current profile and profile 0 every 200 ms. A tag on any profile is therefore always reachable on profile 0 within 200 ms, and the LoRa command path reaches it at any time regardless.
- **Two radios, one truth.** A request or confirm is valid if it arrives on either radio; the tag acknowledges on both. Losing one radio never stalls the handshake, and the LoRa path alone can always drive the tag back to profile 0 at maximum power.
- **Two stations, one tag.** Requests carry the station's PC id; the tag follows the most recent confirmed request and tells both stations its current settings in every frame. A station whose requested settings are not what the tag reports simply adapts its receiver to what the tag reports (the tag's settings are authoritative), and does not fight: it never requests a less robust profile than the other station's last confirmed one.

- **Hard rule for the implementation:** there is no code path on either side that changes a radio setting without a confirmed sequence number, except the reverts toward profile 0. A watchdog on each side asserts, once per second, that the configured radio matches the last confirmed state or profile 0; any other state is reset to profile 0.

9K Two stations, one tag: adaptation is frozen, backoff untouched required

> **Backoff already costs performance today** (§8: 42 + 30 valid ranges/s with two stations against 81–89/s alone, p95 gaps of 83 / 229 ms). Nothing in §9 may make that worse, and nothing in §9 may add a second mechanism that interacts with it. So the rule is not "coordinate the two", it is "do not adapt while there are two".

- **Two stations ⇒ ground state.** While the tag has heard polls from more than one PC id in the last 2 s, it reports that, and every adaptive feature is off on both sides: profile 0, maximum power, `K = 1`, no requests sent, no handshake in flight. A station whose anchors have backoff armed, or that reads "two stations" from the tag, likewise sends nothing but profile 0. The §8 backoff, yields, duty cycle and release logic then run exactly as they do now, on the configuration they were tuned on.
- **Entering coexistence.** If a second station appears while the tag is on a near profile at low power, the tag reverts to profile 0 at maximum power by itself (rule zero's unilateral revert, within one listen-rotation period) and the first station follows the tag's reported settings. The newcomer always polls on profile 0 and so reaches the tag within 200 ms; it never has to know what the tag was doing before.
- **Leaving coexistence.** Adaptation may resume only after the tag has reported a single station for 2 s and no anchor of that station has backoff armed. The first step away from profile 0 is then one ordinary 9J handshake.
- **Anchor-local features that do not touch the air** — 9G LDE preset, the 9A/9C reply bytes, the 9B CIR latch — stay on with two stations. 9F repeats do touch the air and are forced to `K = 1`.
- **The fixed duty cycle stays as tuned.** While sharing the tag the host takes turns on fixed ON/OFF slots and releases a backed-off anchor after ON + 250 ms (§8, host side). Those timings assume one exchange per poll on profile 0; the freeze keeps that true, so the slots, the release margin and the yield are not recomputed for §9 and never need to be. The tag's silence timers (500 ms revert to the previous settings, 1 s to profile 0) must be longer than the longest configurable OFF slot plus the release margin, so a station's own pause can never look like a lost link to the tag; with two stations the tag is on profile 0 anyway, and a lone station never pauses, so duty cycling and adaptation never run at the same time. The only overlap is the hand-over, and it passes through the ground state.
- **Per-station sequence numbers** (9J) remain, so a stale request from a station that has just gone quiet cannot be mistaken for a live one.
- **Regression gate.** Every §9 feature ships only if the §8 two-station bench (both orders of joining, 20 s overlap, one station alone 10 s before and after) shows valid ranges per second and p95 gaps no worse than the sC3.0.70 figures, with the feature enabled on both stations. A feature that costs two-station throughput is kept off until it is fixed, whatever it buys alone.

Bench acceptance: Fly alone at 100 m on profile 0, Mob joins at 2 m: Mob ranges within 1 s on profile 0; the tag reports two stations; neither station sends a request; the §8 rates hold. Mob alone for 2 s after Fly stops: the first handshake to the near profile completes; Fly returns: the tag is back on profile 0 at maximum within one rotation period and Fly ranges within 1 s. 20/20 in each direction.

9I Every §9 feature is switchable and configurable from the app required

Nothing in §9 is always-on. Each feature has its own on/off and its settings in **ADVANCED → Configuration** (persisted in `hitan_settings.cfg` like the backoff and duty-cycle knobs), ships **off**, and is pushed to the anchors by the station worker on start and on every change, like opcode 19 today.

Opcode 22 `SET_LINK_FEATURES` replaces the narrower `SET_TX_POWER_MODE` of 9D:

BYTE	FIELD	MEANING
0	`features` u8 bitmask	bit0 9F repeats · bit1 9G LDE preset · bit2 9D power loop · bit3 9H profile negotiation · bit4 9E channel alternation · bit5 9B CIR window latch · bit6 9A/9C extra reply fields (always safe to leave on)
1	`k_max` u8	9F: 1–5 (0 = anchor decides)
2–3	`lora_target_dbm`, `lora_band_db` i8, u8	9D: target level and half-width (default −70, 6)
4–5	`uwb_target_dbm`, `uwb_band_db` i8, u8	9D: default −80, 5
6	`profile_max` u8	9H: highest profile the anchor may request (0 = never leave profile 0; 1 = mid only; 2 = near allowed)
7	`profile_override` u8	9H: 0xFF = automatic, else pin this profile (bench use)
8	`step_interval_ds` u8	rule zero: minimum time between steps away from robustness, in 0.1 s (default 20 = 2 s)

- **Default is today's behaviour.** With all bits clear the anchor and the tag behave exactly as anc3.1.5 does now: one exchange per poll, LOS LDE, maximum power, profile 0. The tag's own default is profile 0 at maximum power; it is what the tag boots into, falls back to after 1 s without an anchor, and returns to the moment the anchor stops requesting anything else.
- **Off means revert, not freeze.** Clearing bit2 or bit3 makes the anchor send "profile 0, maximum power" on its next polls until the tag echoes it, so switching a feature off from the tablet always lands the tag on its default within a few frames, never leaves it where the loop last put it.
- **STATUS reply** gains a `link_state` block: the active feature mask, the requested and echoed profile, the requested and echoed tag powers, the active LDE preset and the last `k_used`, so the Configuration tab shows what is in force per anchor and the tablet can tell a tag that is not following a request.

- **App side:** a "Link adaptation" group in Configuration with one checkbox per feature and the numbers above as tap-to-edit fields, the per-anchor `link_state` as a live line, and a single **REVERT ALL** key that clears the mask and pins profile 0 — the panic button. Nothing in LOCATE changes; the dial does not know which profile is in force.
- **Recording:** the per-anchor sample gains the echoed profile and tag powers, so a replay can tell a quiet tag from a far one and the hot/cold bar is normalised in playback as it is live.

ORDER	CHANGE	WHAT THE HOST GETS	COST
1	9F	one-sided NLOS error cut by the minimum of repeated exchanges	small, anchor only
2	9G	first-path detection tuned to the state per round	small, anchor only
3	9A	prNlos and the exchange tick, free on every diag read	small, anchor only
4	9C	anchor-side timing; signal RSSI for the arc; the echo bytes 9D/9H need	small, anchor only
5	9D + 9H	linear receivers close in, and the near/mid/far radio profiles (power, rate, preamble, SF, beacon rate, channel) under one negotiated byte	tag + anchor, bench pass for the loop; antenna check for ch 4
6	9B	a real CIR window per range — the NLOS gate the current diagnostics cannot give	anchor, new opcode, opt-in per round
7	9E	fade decorrelation near the ground (a field of the near profile)	both firmwares, channel plan

Bench acceptance.

- **9A:** the DIAGNOSTIC_STATUS reply grows by 8 bytes; prNlos per round near 0 in clean air, toward 1 with a metal box in the path.
- **9B:** the 32-sample window shows the first-path edge within its first 10 samples in clean air; `nlos_features.rs` extended with the window features scores clearly above RX−FP on the same recordings.
- **9C:** the three anchors' `exchange_tick_ms` sit 10–12 ms apart as the worker polls them; `lora_signal_rssi` ≥ `lora_rssi` − noise floor.
- **9F:** tag 3 m behind a metal box: the reported range's median error drops by at least half against K=1 on the same setup; round time grows by no more than 25 ms at K=3.
- **9G:** the STATUS preset bit follows a hand placed over the tag within 10 exchanges and clears within 10 after; no chatter at a steady 7 dB.
- **9H:** walking a tag from 50 m to 1 m, the echoed profile steps 0 → 1 → 2 with no missed round at the switch; pulling the tag away, both sides are back on profile 0 within 1 s of the first missed frame; ch 4 only if the bench shows ≥ 80 % valid exchanges at 10 m.
- **9J:** with the LoRa radio of the tag disabled on the bench, a profile change still completes over UWB alone, and the reverse; with frames dropped at random (20 %) the tag never ends up on a configuration the anchor is not listening on (the STATUS link block and the tag's own report always agree within 1 s); 100/100.
- **9I:** with every feature off the anchor's replies, round time and the tag's power are byte-identical to anc3.1.5 (replay compare); REVERT ALL from the tablet lands the tag on profile 0 at maximum within 3 frames, from any state, 20/20 trials.
- **9D:** tag 1 m from a MOB station: `SignalRssiPkt` settles in −70 ± 6 dBm within 3 s and the Configuration tab's LoRa spread stops reading "compressed"; UWB `rx_power_dbm` settles in −80 ± 5 dBm and the raw bearing scatter at 1 m drops from 12–14° to under 8°. Tag pulled to 50 m: both back at maximum within 1 s of the first missed frame.

§6 Bench verification & at-a-glance

Confirm §3 (RX deafness) in 5 minutes, before changing anything else. After `dwConfig()` runs, read `GPIO_MODE` (0x26:00) bits 19:18 (MSGP6): if they read **00** despite `dwt_setlnapamode` writing **01**, the GPIO write silently failed — that is the smoking gun. Also read `PMSC_CTRL0` bits 16/17 (expect **0/0** now, **1/1** after 3A). Scope GPIO6/pin 30 during RX (flat-low now → pulses high after 3A). Then apply 3A and re-run Skylab TX → Rev-D RX at 400 m. If loss drops toward 0–4%, confirmed firmware-fixable; if not, the deficit is in Rev-D's passive RX front-end (hardware).

DO FIRST (BLOCKERS)

- **3A** — GPIO clocks → RX LNA *(the Rev-D fix)*
- **1A** — 110k carrier-int multiplier
- **1B** — reject negative ToF
- **1C–1F** — BR_110K + PLEN_2048/PAC64 + sfdTO, both ends
- **8A / 8B** — backoff: arm once; CTRL=0 releases *(two stations)*
- **5E** — UART error callback re-arms RX (deaf anchor)

THEN (RANGE MARGIN / MULTI-STATION)

- **2A** — TX-1 errata clock enable
- **4A** — TX power temp compensation
- **1G / 1H** — RX / preamble-detect timeouts
- **4B** — per-device antenna cal
- **8E** — seed backoff RNG
- **8F** — imposter id mask (latent; PC id ≥ 8192)

NEXT (§9, HOST REQUEST 2026-10-21)

- **9F** — repeat exchanges, report the minimum (anchor only)
- **9G** — LDE preset per state (anchor only)
- **9A** — latch LDE_PPINDX + tick in the diag reply
- **9C** — exchange tick, signal RSSI, tag TX echo in the TWR reply
- **9D / 9H** — negotiated TX power and link profile, tag + anchor
- **9J** — request / ack / confirm / proof handshake, seq numbers, both radios, unilateral revert
- **9K** — two stations: adaptation frozen on profile 0, backoff untouched, §8 figures as regression gate
- **9I** — every feature switchable from the app, default off, REVERT ALL
- **9B** — CIR window, opcode 21

VERIFY / WHEN SCALING

- **5A** — clean status-bit clearing
- **5B** — `==` / `<=` consistency
- **8D** — backoff window: coarse vs fine *(needs 8A)*
- **7A** — LDE NTM/PMULT sweep *(after §3)*

Verdict on the TWR logic: sound. The directional A/B result (Rev-D→Skylab works, Skylab→Rev-D fails) is itself proof that the SS-TWR exchange, addressing, and timing are correct — a logic defect would be symmetric. The accuracy items (1A, 4B) affect distance *values*, not whether frames are received; the genuine logic nits (5B, 5C) don't cause the long-range loss. The long-range loss is a Rev-D *receive* problem — §3.

Line numbers reference `documentation/fw_actual_ref/`. Grounded in: DW1000 User Manual §7.2.39 / §7.2.40.11 / §7.2.47.2 / Tables 6, 21, 57 · DW1000 Errata §3.1.3 · APS004 §2–3 · APS006 Pt 2/3 · APS011 · APS014 · APS017 · APS023 Pt 1 §8.2. The reference tree is read-only; apply these in your working firmware source.